



WEST UNIVERSITY OF TIMIȘOARA
FACULTY OF COMPUTER SCIENCE
BACHELOR STUDY PROGRAM: ARTIFICIAL
INTELLIGENCE

BACHELOR THESIS

SUPERVISOR:

Lect. Dr. Spătaru Florin
Adrian

GRADUATE:

Băbțan Gelu Alexandru

TIMIȘOARA
2026

WEST UNIVERSITY OF TIMIȘOARA
FACULTY OF COMPUTER SCIENCE
BACHELOR STUDY PROGRAM: ARTIFICIAL
INTELLIGENCE

PageSageAI - AI-Powered STEM Teaching Assistant in a Virtual Classroom Environment

SUPERVISOR:

Lect. Dr. Spătaru Florin
Adrian

GRADUATE:

Băbțan Gelu Alexandru

TIMIȘOARA
2026

Abstract

In the context of the increased usage of artificial intelligence in recent years and the spread of Large Language Models (LLMs) in many domains, including education, the expectation for chatbots and teaching assistants has increased significantly. Nevertheless, finding an educational LLM that can perform all the functions of a real teacher remains a challenge for students.

In this paper, we introduce a new approach to online teaching that combines the main functionalities of educational LLMs to create a structural, unified and comprehensive alternative to existing learning assistants. The presented system combines existing LLMs and TTS models with Retrieval-Augmented Generation (RAG) techniques to parse PDF course materials and deliver all the information in audio format. Another feature is that the student can ask any question to a chat bot during the audio teaching process. It can also generate short explanatory videos showing algorithm execution, mathematical functions or any STEM concept that can be shown visually, alongside evaluation test at the end of the teaching process. The main focus is on user–assistant interaction, making the learning process as interactive as possible and helping all types of students to understand and synthesise the provided lectures. Our final product is a virtual classroom platform that enhances and eases the learning process, making it more accessible for anyone eager to learn.

Contents

Abstract	iii
1 Introduction	1
1.1 Application Context	1
1.2 Project Motivation	2
1.3 Application Scope	3
1.4 Project Objectives	3
2 Related Work	4
2.1 AI-Powered Educational Assistants	4
2.2 Automated Visualization Generation	5
2.3 Comparative Analysis and Positioning	6
3 System Design, Implementation and Requirements	7
3.1 Requirements	8
3.1.1 Functional Requirements	8
3.1.2 Non-Functional Requirements	9
3.2 System Architecture	10
3.2.1 High-Level Diagram	10
3.2.2 Technology Stack	10
3.2.3 System Components	11
3.3 Modules Design	12
3.3.1 Module 1: Authentication Module	12
3.3.2 Module 2: Class, Enrollment, Assignment & Submission Module	15
3.3.3 Module 3: PDF Ingestion & Narration Generation Module . .	17
3.3.4 Module 4: RAG Indexing, Query & ChromaDB Seeding Module	19
3.3.5 Module 5: Concept Animation Module	21
3.3.6 Module 6: Quiz Module	24
3.4 Databases Design	25
3.4.1 Schema Design	26
3.4.2 Relational Model (PostgreSQL)	26
3.4.3 Vector Store (ChromaDB)	27
4 System Evaluation	28
4.1 API Performance Testing	28
4.2 Load Testing	28
4.3 Pipeline Reliability	28

4.4	Security Testing	28
5	Empirical Evaluation	28
5.1	Overview	28
5.2	Study Objectives	30
5.3	Participants	31
5.4	Materials and Instruments	31
5.4.1	Source Material	31
5.4.2	Instructional conditions	32
5.4.3	Questionnaire and Procedure	32
5.5	Results	33
5.5.1	Descriptive Statistics	33
5.6	Discussion	34
5.6.1	Interpretation of Results	34
5.6.2	Limitations	35
5.6.3	Conclusions	36
6	Conclusion and Future Work	36
6.1	Conclusion	36
6.2	Future Work	36

1 Introduction

The growth of Artificial Intelligence has the power to personalize and revolutionize the way people will learn in the future. The rapid spread of Large Language Models (LLMs) across numerous domains has opened new possibilities for educational technology, shifting the focus from static, towards adaptive, interactive learning experiences. This chapter establishes the context and motivation behind the proposed system. Section 1.1 outlines the broader landscape of AI in education and its evolution. Section 1.2 presents the specific problems that motivate this work. Section 1.3 defines the scope of the application and positions it against existing tools. Section 1.4 lists the concrete objectives the system is designed to achieve.

1.1 Application Context

The integration of Artificial Intelligence into education represents one of the most significant shifts in the history of learning and teaching. AI in education refers to the use of technologies such as machine learning and natural language processing to enhance the learning experience through algorithms that analyse data, identify patterns, and generate predictions [1]. This definition captures the core value proposition: AI does not merely digitise existing instruction — it fundamentally changes what is possible within it.

The evolution of AI in education spans three distinct paradigms. The first, emerging in the 1970s and 1980s, was defined by rule-based Intelligent Tutoring Systems (ITS) such as TICCIT and SCHOLAR, which attempted to replicate one-on-one tutoring through predetermined decision trees [2]. The second paradigm, spanning the 1990s to 2010s, introduced machine learning and educational data mining, enabling platforms such as Cognitive Tutor and ALEKS to model student knowledge states and adapt content delivery accordingly. Despite their effectiveness in controlled settings, these systems remained constrained to narrow, structured domains and failed to engage students in natural dialogue. The third and current paradigm, inaugurated by the release of large-scale transformer models and culminating in GPT-4 and similar systems, has shifted the field from rule-based automation to genuine cognitive partnership — systems capable of explaining, questioning, generating, and adapting in real time across any domain [3].

The practical impact of AI on educational outcomes is increasingly well-documented. AI technologies have been shown to enhance knowledge retention rates by up to 30% (aici in pdf nu imi apare continuarea dupa ')

These benefits derive from several distinct mechanisms. First, AI enables per-

sonalisation at scale: unlike a single instructor managing a class of thirty or more students, an AI system can simultaneously adapt content, pacing, and difficulty to each individual learner’s knowledge state and learning style [1, 4]. Second, AI-powered systems provide immediate, formative feedback — a factor consistently linked to improved learning outcomes in the literature [5]. Third, AI removes temporal and geographic barriers to quality instruction, making educational support available on demand rather than constrained to scheduled class hours. Fourth, AI-driven tools have demonstrated significant capacity to enhance student engagement through interactive, adaptive experiences that static materials cannot replicate [6].

Nowhere is this transformation more urgently needed than in STEM education. The inherently abstract and dynamic nature of STEM concepts — algorithm execution, function behaviour, data structure manipulation, mathematical proof — places exceptional demands on visual and interactive explanation. Research in multimedia learning consistently demonstrates that students comprehend abstract processes more effectively when verbal explanations are accompanied by dynamic visual representations rather than static text or diagrams [7]. Yet the majority of existing AI educational tools address only isolated dimensions of the learning process: answering questions, summarising materials, or generating assessments in isolation. The digitalisation of education has created the infrastructure for a complete, unified AI teaching experience — but the tools to deliver one remain largely absent from the current landscape.

1.2 Project Motivation

One of the most effective ways to understand and learn technical concepts is through visual representation. A major problem with STEM courses is that the visual concepts and figures presented by teachers are usually very general and static. Studies comparing dynamic and static visualisations consistently report a significant advantage of instructional animations over static pictures, with a mean weighted effect size of $d = 0.37$ on learning outcomes [8], rising to $d = 1.06$ when procedural or algorithmic knowledge is to be acquired [8]. Research conducted across middle and high school students further confirms that dynamic visualisations such as 3D models and animations produce significantly higher learning outcomes and intrinsic motivation compared to static equivalents [9]. The possibility of interacting with and understanding edge cases, and of visualising main concepts evolving over time, becomes less intuitive for a large proportion of students when instruction relies exclusively on fixed diagrams and slides.

Another problematic point is the lack of one-to-one interaction between teachers and students during lectures. As documented by the OECD [10], demographic shifts, teacher shortages, and budget constraints continue to widen the gap between the number of students and the available instructional support per learner, meaning that self-directed learning becomes part of most students’ academic lives. However,

not all students possess the metacognitive skills required to independently interpret core concepts and foundational information [11]. In addition, teachers could benefit from knowing the comprehension level of their classes after each session, so that instruction can be adjusted to address knowledge gaps in a timely manner.

1.3 Application Scope

In this context, an AI-powered Online Teaching Assistant can address the limitations of both traditional instruction and existing educational tools by combining personalised content delivery, on-demand interaction, visual concept explanation, and automated assessment within a single coherent environment. This paper develops and evaluates a Generative AI based Teaching Assistant integrated into a virtual classroom platform, aimed at demonstrating the potential of artificial intelligence to replicate a complete, structured university teaching session grounded in course-specific material.

Despite the rapid growth of AI in education, the current landscape remains largely fragmented. Existing tools tend to address isolated aspects of the learning process in isolation — tutoring, summarisation, or assessment — rather than integrating them into a unified experience [12, 13]. For decades, educational technology functioned through separate, disconnected applications, each serving its individual purpose while creating digital silos that forced students to constantly switch context and lose the continuity essential to effective learning [14]. As a result, the full teaching lifecycle — structured content delivery, live student interaction, visual explanation, and comprehension verification — has remained largely unaddressed by any single platform [6].

The proposed system bridges this gap by unifying all dimensions of the teaching experience into one platform: a teacher uploads course material, which is processed and delivered as a structured vocal lecture; the student can interrupt at any point to ask questions answered by a RAG-grounded module; on-demand concept animations are generated for abstract or visual STEM topics; and an automated quiz evaluates comprehension at the end of each session. This end-to-end approach is designed for university-level STEM courses, where the combination of abstract content, high cognitive load, and limited one-to-one instructional support makes the need for intelligent, integrated assistance most acute [15].

1.4 Project Objectives

The main objectives of this thesis are:

1. To design and implement a vocal AI teaching agent capable of delivering reformulated university course content derived from PDF sources in a natural, pedagogically structured manner.

2. To implement a Retrieval-Augmented Generation (RAG) based question answering module that grounds answers in both the original course material and the general content related to the subject.
3. To integrate and maintain a code generation model capable of producing Manim animation code from natural language concept descriptions, enabling dynamic visual representations of STEM topics.
4. To integrate automatic quiz and test generation tied to lecture content, providing immediate evaluation after each teaching flow.
5. To build a virtual classroom platform that unifies all AI components under a structured teacher–student role system, supporting course management, live AI tutoring sessions, and student progress tracking.

2 Related Work

The development of the proposed system is built on a range of existing technologies and platforms in the field of AI-powered education. This chapter reviews the most relevant work in the area: existing AI teaching assistants and their capabilities, and a comparative positioning of the proposed solution against the tools currently available on the market.

2.1 AI-Powered Educational Assistants

Khanmigo, developed by Khan Academy and built on GPT-4 [16, 17], is one of the more widely used AI tutoring tools available today. The way it works is quite direct: instead of giving students the answer directly, it asks them questions and offers hints, pushing them to work through the problem themselves. This makes it effective for building reasoning skills, and on the teacher side it covers things like lesson planning, quiz creation, and rubric generation. The main problem with Khanmigo for university use is that it has no connection to external course materials. It does not accept PDF uploads, so there is no way to point it at a specific professor’s slides or textbook and have it teach from there. Any question a student asks gets answered from general knowledge, with no awareness of what was actually covered in the course.

NotebookLM, developed by Google and powered by Gemini 1.5 Pro [18, 19], takes a document-centred approach to learning. Users can upload lecture materials such as PDFs and the platform produces text summaries, audio podcast-style overviews, and quizzes drawn from the uploaded content. Unlike Khanmigo, NotebookLM does

engage directly with uploaded source material, which makes it more relevant for students working with specific course documents. However, its features are still lacking consistency, being oriented towards individual research and study rather than structured teaching. The audio overviews it generates are one-time, non-interactive summaries — a student cannot pause them, ask a question mid-way through, or request a concept to be explained differently. The quizzes it generates derived from the document are also not sufficiently specific given the current development state of the application. These characteristics make NotebookLM a useful study companion but not a replacement for a live teaching experience.

SchoolAI [20] runs on GPT-4o and GPT-4.1 and is built mainly for K–12 classrooms. Teachers set up structured activities called “Spaces” where students work through topics with AI guidance, and the platform lets teachers monitor what is happening in real time and keep the interactions within appropriate boundaries. The interaction model is reactive though — the AI waits for the student to say something and responds to that, rather than taking the initiative to deliver content. SchoolAI also has limited support for external document ingestion, and its K–12 focus makes it a poor fit for the depth of university-level STEM material. No dynamic visualisation is generated at any point in the interaction.

All three platforms have genuine strengths in their respective areas. The problem is not that they are poorly built — it is that none of them was designed to carry a student through an entire university lecture from start to finish, grounded in the teacher’s own course material, with room to ask questions along the way and see concepts demonstrated visually. That gap is what the proposed system is built to fill.

2.2 Automated Visualization Generation

The challenge of making STEM concepts visually accessible is not new. Educators have long relied on diagrams, physical demonstrations, and more recently digital animations to illustrate abstract processes. Among the tools that have attempted to address this in a digital context, Manim stands out as the most capable open-source framework for programmatic educational animation. Manim was originally developed by Grant Sanderson [21], the mathematician and educator behind the YouTube channel 3Blue1Brown, as a personal tool for creating the mathematically precise animations his channel became known for. The community edition, ManimCE, was subsequently forked from Sanderson’s repository and has since been maintained and extended by an active open-source community. The framework is built on Python and uses a scene-based architecture in which visual objects — geometric shapes, function graphs, text, LaTeX equations — are defined programmatically and animated using a set of built-in transformation methods. This approach gives educators precise control over every aspect of an animation, from timing and colour to the step-by-step progression of a concept. The framework contains many built-

in functions that naturally create animations of objects and embeds LaTeX for writing mathematical equations, allowing users to create clear, professional-looking animations with minimal effort.

Previous implementations demonstrate this across a range of STEM topics, showing Manim applied to heapsort, tree data structures, the Gaussian integral, and geometric transformations. The breadth of these examples illustrates that the framework is not limited to any single domain — it is general enough to handle both algorithmic and mathematical content, which makes it well suited to university-level STEM courses that span both areas.

Despite its power, Manim has a significant accessibility barrier: it requires the user to write Python code. While creating Manim animations often only requires knowledge of a small subset of Python syntax, this still places it out of reach for the majority of students and for educators without a programming background. NotebookLM, the closest existing tool to document-based teaching, does generate some video content from uploaded materials, but these are pre-structured overviews rather than concept-specific animations. A student asking how merge sort works will receive a text or audio explanation — not a frame-by-frame visual walkthrough of the algorithm executing on an example array.

The proposed system bridges this gap by integrating a large language model capable of generating Manim code from natural language descriptions. When a student requests a visualisation during a tutoring session — for example, asking to see how a binary search tree insertion works — the system constructs a structured prompt from the student’s request and a relevant context-based system prompt, passes it through the Gemini API, and executes the resulting Python code on the server. The rendered animation is then delivered directly within the classroom interface. This pipeline removes the requirement for any programming knowledge on the part of the student, making dynamic STEM visualisation accessible on demand for the first time alongside an integrated teaching platform.

2.3 Comparative Analysis and Positioning

The table below summarises the key capabilities of the three reviewed platforms alongside the proposed system across the dimensions most relevant to a complete AI teaching experience.

Table 2.1: Comparison of existing AI teaching tools with the proposed system.

System	Vocal Teaching	PDF Grounding	Dynamic Viz.	Quiz Gen.	Classroom Mgmt.
Khanmigo	No	No	No	Yes	Yes
NotebookLM	Partial	Yes	Partial	Yes	Yes
SchoolAI	No	Partial	No	Yes	Yes
Proposed System	Yes	Yes	Yes	Yes	Yes

As Table 2.1 shows, no single existing platform covers all five dimensions. Khanmigo offers strong student interaction but is entirely disconnected from external course materials and generates no visual content. NotebookLM engages with uploaded documents and produces audio content, but its results are non-interactive — a student cannot interrupt it, ask a question, or receive a custom visualisation during the teaching process. SchoolAI provides some classroom management features and supervised AI interaction, but its content is not grounded in uploaded course material and no dynamic visualisation is available at any point.

The proposed system is the only one among those evaluated that combines structured vocal lecture delivery from a teacher-uploaded PDF, real-time student interruption with RAG-grounded answers, on-demand Manim animation generation, automated quiz creation, and a full classroom management layer for teachers and students. Rather than assembling these features from separate tools, the system integrates them into a single environment built around the full lifecycle of a university teaching session.

3 System Design, Implementation and Requirements

The design and implementation of PageSageAI follows established software engineering principles, structuring the system around a clear separation between what the system must do and how well it must do it. According to Sommerville [22], a well-formed software specification distinguishes between functional requirements — which describe the services and behaviours the system must provide — and non-functional requirements, which constrain the qualities of those services such as performance, security, and maintainability. This chapter applies that structure to define the complete requirement space of the platform, followed by the architectural and module-level design decisions that realise it.

3.1 Requirements

3.1.1 Functional Requirements

Functional requirements specify the concrete capabilities the system must expose to each class of user [22]. For PageSageAI, these are organized around the two primary roles — teacher and student — and the AI pipelines that serve both, covering the full lifecycle from course upload to final assessment.

- **Teacher Dashboard:** Allows educators to create and manage classes, enroll or remove students, upload PDF course materials, post and grade assignments, send announcements to students, and monitor comprehensive per-student progress metrics, including the ability to trigger AI-generated quizzes for any uploaded course.
- **Student Dashboard:** Allows students to enroll in classes using a class code, submit and resubmit assignments, view grades and feedback, and access a fully AI-assisted learning experience for each uploaded course: vocal teaching synchronized with the PDF view, live question answering during the lecture, on-demand visual representations of concepts, and a quiz system for self-assessment and review.
- **PDF Parsing and Content Extraction:** Converts uploaded PDF course materials into structured, page-by-page content — extracting both text and embedded images — which forms the basis for the narrated teaching session, ensuring that every page, including its visual content, is represented in the final lecture.
- **Smart Vocal Tutoring:** Transforms the extracted page content into natural-language narration and streams it to the student as synthesized speech via text-to-speech, with narration generated to specifically describe and reference the images, diagrams, and concepts present on each page as the student progresses through the course.
- **Dynamic Visualization:** Allows students to describe any concept encountered in the lecture in their own words and have it automatically rendered as a short animated explanation (up to 30 seconds), generated on demand and cached for reuse if the same concept is requested again.
- **Question Answering During Teaching:** Allows students to interrupt the vocal lecture at any point and ask a free-form question related to the subject matter; the answer is generated using a combination of the current lecture’s content, a RAG knowledge base, and the AI model’s general knowledge, ensuring contextually grounded responses with the possibility of regeneration.

- **Automated Evaluation Pipeline:** Automatically generates a multiple-choice quiz based on the full content of the lecture once it has been processed, with each question validated for structural correctness; teachers can review the generated quiz and trigger regeneration if needed before making it available to students.

3.1.2 Non-Functional Requirements

Non-functional requirements define the quality attributes a system must satisfy alongside its functional behaviour [23, 24]. Following the ISO/IEC 25010 quality model [25], the platform is evaluated across five key dimensions: performance, scalability, security, reliability, and maintainability.

- **Performance:** Standard API requests must respond within 2 seconds under normal load. All long-running AI operations (PDF extraction, narration, TTS synthesis, ChromaDB indexing, Manim rendering, quiz generation) run as background daemon threads, so the triggering request returns immediately while the frontend polls for status. Media assets are served directly by Django’s media handler, and shared React context caches frequently reused data to minimise redundant network requests.
- **Scalability:** Each AI pipeline step runs in its own daemon thread, allowing multiple courses to be processed in parallel without blocking one another. The shared ChromaDB instance supports multiple course collections simultaneously, with chunks prefixed by course and page number to prevent collisions.
- **Security:** Authentication uses JWT tokens via `djangorestframework-simplejwt` with 5-minute access tokens and daily refresh tokens. All protected endpoints require a valid Bearer token. Access control is enforced at the view level through ownership-based ORM queries rather than generic permission classes. CORS is restricted to the frontend origin, file uploads are stored in type-segmented media directories, and passwords are hashed by Django’s authentication framework.
- **Reliability:** AI pipelines are idempotent — each step checks for existing output before re-executing, enabling safe resumption after crashes. Courses left in `PROCESSING` state on server startup are automatically resumed. Animation generation retries up to 3 times before being marked as failed. Pipeline errors are logged in a consistent format and persisted to the database for meaningful UI reporting.
- **Maintainability:** The codebase separates models, views, serializers, pipeline modules, and business-rule constants into dedicated files organised by domain. All AI logic is isolated in pipeline files, decoupled from the request/response cycle, making each pipeline independently testable and replaceable. Fixed

business rules are centralised in a constants module rather than scattered as magic numbers across the codebase.

3.2 System Architecture

The orchestration pattern establishes a decouple layer dividing presentation nodes, analytical processing structures, and heavy generation workers.

3.2.1 High-Level Diagram

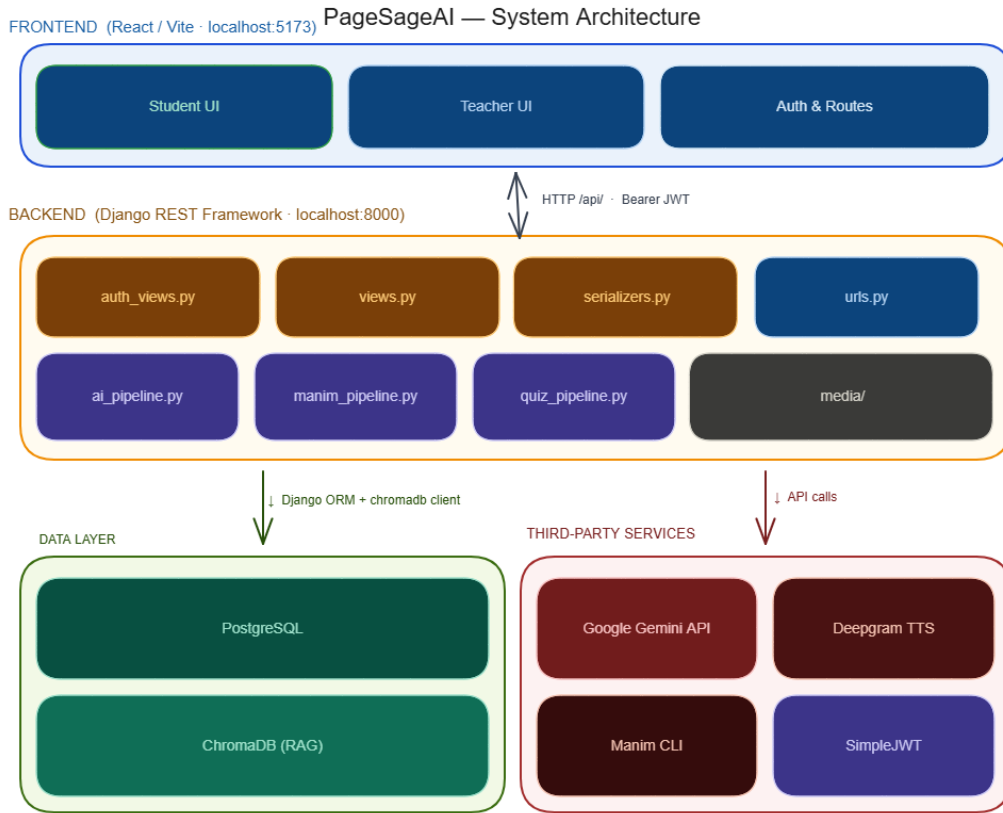


Figure 3.1: General System Architecture (High-Level System Architecture).

3.2.2 Technology Stack

- **Frontend:** React.js with Vite (JSX), React Router v6, JWT authentication stored in `localStorage` with role-based protected routes, Fetch API for server communication, polling for asynchronous AI pipeline monitoring, and an `AssignmentsContext` for client-side data caching.
- **Backend:** Django with Django REST Framework (DRF), authentication

via `djangorestframework-simplejwt` (5-minute access tokens, daily refresh tokens, payload extended with role and first name). Long-running AI tasks run in background daemon threads, are idempotent, and are recoverable after restart via `ApiConfig.ready()`. Settings are split into `base.py`, `local.py`, and `production.py`. CORS is restricted to the frontend origin.

- **Databases:** PostgreSQL for relational data (users, classes, enrollments, assignments, submissions, AI courses, quizzes) via Django ORM, with a custom `User` model using email as the primary identifier and a `role` field (`student` | `teacher`). ChromaDB stores vector embeddings for RAG, generated locally with `all-MiniLM-L6-v2` Sentence-BERT [26] (chunk size 700 tokens, overlap 100 tokens).
- **External APIs and Services:** Groq (`llama-3.1-8b-instant`) for low-latency narration text generation; Google Gemini (`gemini-2.5-flash`) for RAG synthesis, live Q&A, Manim code generation, and quiz production, with two API keys for rate-limit isolation; Deepgram Aura-2 for text-to-speech MP3 conversion; Manim Community Edition invoked as a local subprocess (`manim -q1`) to render Python scene scripts into MP4 videos, with up to three automatic retries.

3.2.3 System Components

- **API Core Service:** A Django REST Framework application exposing role-scoped endpoints for classes, enrollments, announcements, assignments and submissions, and AI-teaching features; ownership and role checks (student vs. teacher) are enforced explicitly in the view layer rather than through generic permission classes.
- **AI Pipeline Services:** Three independent background pipelines, each running in a daemon thread and tracked through an explicit status field on its model: the *AICourse pipeline* (PDF extraction → narration → text-to-speech → ChromaDB indexing), the *ConceptAnimation pipeline* (Manim script generation → rendering), and the *Quiz pipeline* (narration aggregation → multiple-choice question generation). Each step is idempotent and resumable, allowing the application to recover `PROCESSING` courses after a server restart.
- **RAG Retrieval Worker:** Splits page narration text into overlapping chunks (700 characters, 100-character overlap), embeds them locally with `all-MiniLM-L6-v2`, and stores them in a shared ChromaDB collection alongside the pre-seeded OSSU curriculum corpus. At query time, the top- k ($k = 4$) most similar chunks are retrieved and passed to Gemini as context for answer generation.
- **Manim Rendering Subprocess:** A local subprocess invocation of the Manim CLI that compiles an LLM-generated `ConceptScene` Python script into a low-

quality (-q1) MP4 file within a bounded timeout, with no preview window and no external assets, keeping the Django process itself free of rendering load.

3.3 Modules Design

3.3.1 Module 1: Authentication Module

- **Purpose:** Manages user identity across the entire platform. Responsible for registering new users, validating credentials, issuing JWT tokens, refreshing expired access tokens, and enforcing role-based access control on both the frontend and backend. Ensures that student and teacher routes are fully separated and that unauthenticated requests never reach protected resources.
- **Inputs:**
 - POST/api/auth/signup/ — {email,password,firstName,lastName,role}
 - POST/api/auth/login/ — {email,password,role}
 - POST/api/auth/token/refresh/ — {refresh}
 - GET/api/auth/me/ — Authorization:Bearer<token>
- **Outputs:**
 - On signup/login: {token,refresh,user:{id,email,firstName,lastName,role}}
 - On /me: {user:{id,email,firstName,lastName,role}}
 - On refresh: {access}
 - On failure: HTTP 400 / 401 with error message
- **Dependencies:**
 - *Internal:* api/models.py (User model with email-based auth and role field); core/settings.py (SIMPLE_JWT config: access = 5 min, refresh = 1 day)
 - *External:* django-rest-framework-simplejwt for token generation and validation; AuthContext.jsx storing tokens in localStorage and exposing user state globally; ProtectedRoute.jsx redirecting unauthenticated users to /login
- **Flow Description:** The user submits credentials via the login form. A POST/api/auth/login/ request is sent to the backend; on invalid credentials an HTTP 401 is returned. On success, an access token (5 min TTL) and a

refresh token (1 day TTL) are issued and stored in `localStorage` by `AuthContext`. The role embedded in the token payload determines routing: students are directed to `/student/*` routes and teachers to `/teacher/*` routes, each guarded by `ProtectedRoute`. Rather than reacting to 401 responses, `AuthContext` uses a *proactive* refresh strategy: immediately after storing a new access token, `scheduleRefresh()` decodes its `exp` claim and schedules a silent `POST/api/auth/token/refresh/` call 30 seconds before the token expires. On success the new access token replaces the old one in `localStorage` and the timer is rescheduled for the next cycle; on failure (refresh token expired) `logout()` is called, clearing local storage and redirecting to `/login`. The flow is illustrated in Figure 3.2.

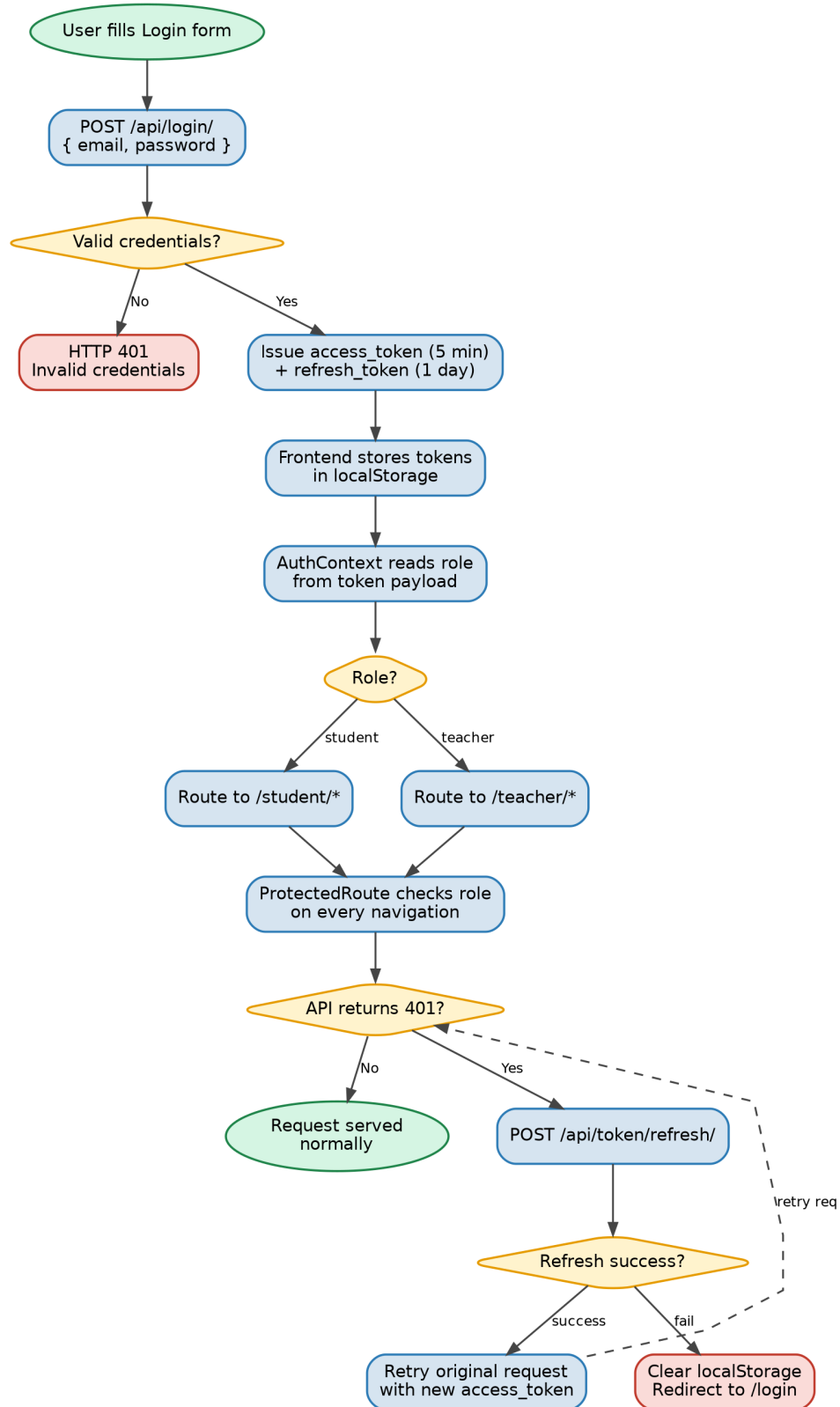


Figure 3.2: Authentication Module

3.3.2 Module 2: Class, Enrollment, Assignment & Submission Module

- **Purpose:** Provides the organizational backbone of the platform. The class and enrollment sub-module allows teachers to create classes (each with a unique enrollment code) and students to discover and join them. The assignment and submission sub-module manages the full lifecycle of academic assignments: creation, file attachment, student submission, resubmission, unsubmit, and grading with denormalized status tracking.
- **Inputs:**
 - *Teacher — classes:* POST/api/teacher/classes/ {name,teacher_name,color,icon}; DELETE/api/teacher/classes/<id>/
 - *Student — classes:* GET/api/classes/; POST/api/classes/enroll/ {code}; DELETE/api/classes/<id>/
 - *Teacher — assignments:* POST/api/teacher/classes/<id>/assignments/ {title,description,due_date} (attachments included as files in the same multipart request); PATCH/api/teacher/classes/<id>/submissions/<sid>/grade/ {grade,note}
 - *Student — assignments:* GET/api/assignments/; POST/api/assignments/<id>/submit/ (file upload); DELETE/api/assignments/<id>/unsubmit/
- **Outputs:**
 - Class list for teacher / student; created class object with 6-character enrollment code
 - Enrollment confirmation or HTTP 404 if code invalid
 - Assignment objects with status: `pending|submitted|graded`
 - Submission objects with uploaded files; grade and note after teacher review
- **Dependencies:**
 - *Internal:* `api/models.py` (Class, Enrollment, Assignment, AssignmentAttachment, Submission, SubmissionFile models); `api/serializers.py` (teacher vs. student variants); JWT middleware for ownership checks; `media/submissions/` and `media/attachments/` for file storage; `AssignmentsContext.jsx` frontend cache
 - *External:* Django file handling (`InMemoryUploadedFile`)
- **Flow Description:** A teacher creates a class via POST/api/teacher/classes/; the system generates a unique 6-character enrollment code and persists the class to PostgreSQL. The teacher shares the code with students, who

submit it via `POST/api/classes/enroll/`. If valid, an Enrollment record linking student to class is created; otherwise HTTP 404 is returned. For assignments, the teacher posts to `/api/teacher/classes/<id>/assignments/`; the assignment is saved with status `pending` and becomes visible in the student dashboard. The student uploads files via `/api/assignments/<id>/submit/`, setting the status to `submitted`. The teacher reviews and grades via `/api/teacher/classes/<id>/submissions/<sid>/grade/`, transitioning the status to `graded`. A student may resubmit at any point, resetting the status to `submitted`. The flows are illustrated in Figure 3.3.

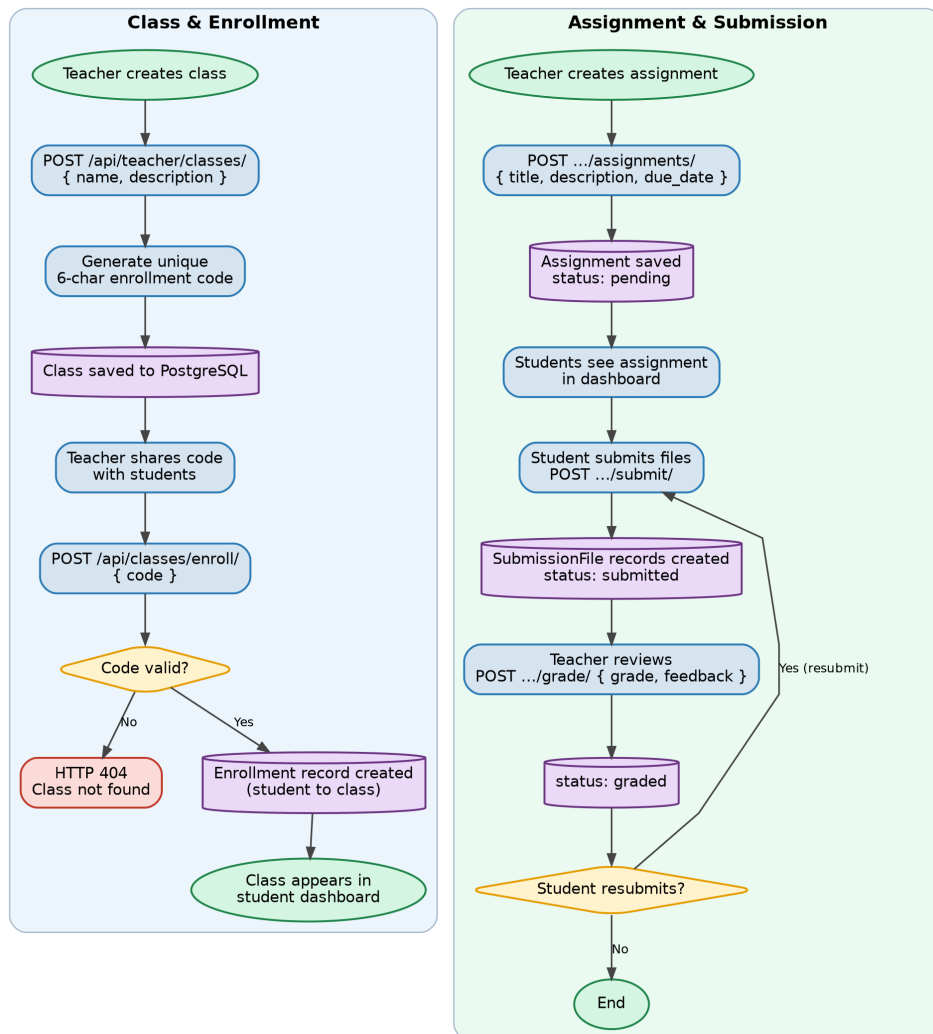


Figure 3.3: Class, Enrollment, Assignment & Submission Module

3.3.3 Module 3: PDF Ingestion & Narration Generation Module

- **Purpose:** Transforms a raw PDF course file into a structured, per-page representation and then converts that structure into natural-language narration text as if spoken by a university professor. The ingestion sub-module renders each page as a high-resolution PNG, extracts text blocks with spatial coordinates, and classifies each block semantically using a local transformer model. The narration sub-module processes pages sequentially, building LLM prompts from classified blocks and calling the Groq API for narration text, with idempotent resume support.
- **Inputs:**
 - *PDF Ingestion:* `pdf_path` — absolute path to uploaded PDF; `output_dir` — target directory for output files
 - *Narration:* `output_dir` containing `page_XXXX.json` files; `model` (default: `llama-3.1-8b-instant`); `api_key` (Groq); `seconds_per_slide` (default: `None` — speaking duration computed automatically per page from content, clamped to 20–110 s.)
- **Outputs:**
 - `page_0001.png ... page_XXXX.png` — rendered page images at 200 DPI
 - `page_0001.json ... page_XXXX.json` — structured block data `{page_number, png_path, width_px, height_px, has_visual_candidate, raw_text, blocks: [{type, text, order, bbox, confidence}]}`
 - `course_index.json` — `{pdf_path, total_pages, pages: [{page_number, png_path, json_path, has_visual_candidate, block_types}]}`
 - `page_0001_narration.json ... page_XXXX_narration.json` — `{page_number, narration_text, word_count, estimated_seconds, target_seconds, target_mode, model_used, blocks_used, has_visual_candidate}`
 - `course_narration.json` — array of all per-page narration results, written at the end of `narrate_course()`
- **Dependencies:**
 - *External (ingestion):* PyMuPDF (`fitz`) [27] for PDF rendering and text extraction; Pillow for RGB image conversion; HuggingFace Transformers [28] (`LayoutLMv3Processor`, `LayoutLMv3ForTokenClassification`); PyTorch for inference; `microsoft/layoutlmv3-base` model [29] (~900 MB, auto-downloaded); google-genai SDK with `gemini-2.5-flash` [30] for figure description (vision)

- *External (narration)*: Groq Python SDK; `llama-3.1-8b-instant` [31] hosted on Groq for lecture narration;
- **Flow Description:** For each page in the PDF, the ingestion sub-module renders a 200 DPI PNG, extracts text spans with bounding-box coordinates, normalises boxes to the 0–1000 LayoutLMv3 scale, and runs transformer inference (token limit: 512). A majority-vote per block produces a semantic type (`title|paragraph|formula|figure|bullet`); heuristics override the model for formulas and titles using font size, bold flag, and text content. For blocks classified as figures, the cropped image region is sent to `gemini-2.5-flash` (vision) to produce a factual 1–3 sentence description, which is stored as the block’s `text` so the narration sub-module can reference it verbally. Results are written to `page_XXXX.json` and a `course_index.json` manifest is saved. The narration sub-module reads each JSON in page order. Speaking duration is computed per page by `estimate_target_seconds()`: approximately 1 second per 14 characters of text, plus a 15-second bonus for pages containing formulas and a 10-second bonus for pages with a described figure; the result is clamped to 20–110 seconds. A professor-style prompt is built from the classified blocks and sent to the Groq API (`llama-3.1-8b-instant`). On rate-limit errors the function parses the exact wait time from Groq’s error message and retries up to 5 times. The narration result is written to `page_XXXX_narration.json`; pages for which this file already exists are skipped, making the process idempotent and resumable. After all pages are processed, the combined results are written to `course_narration.json`. The flow is illustrated in Figure 3.4.

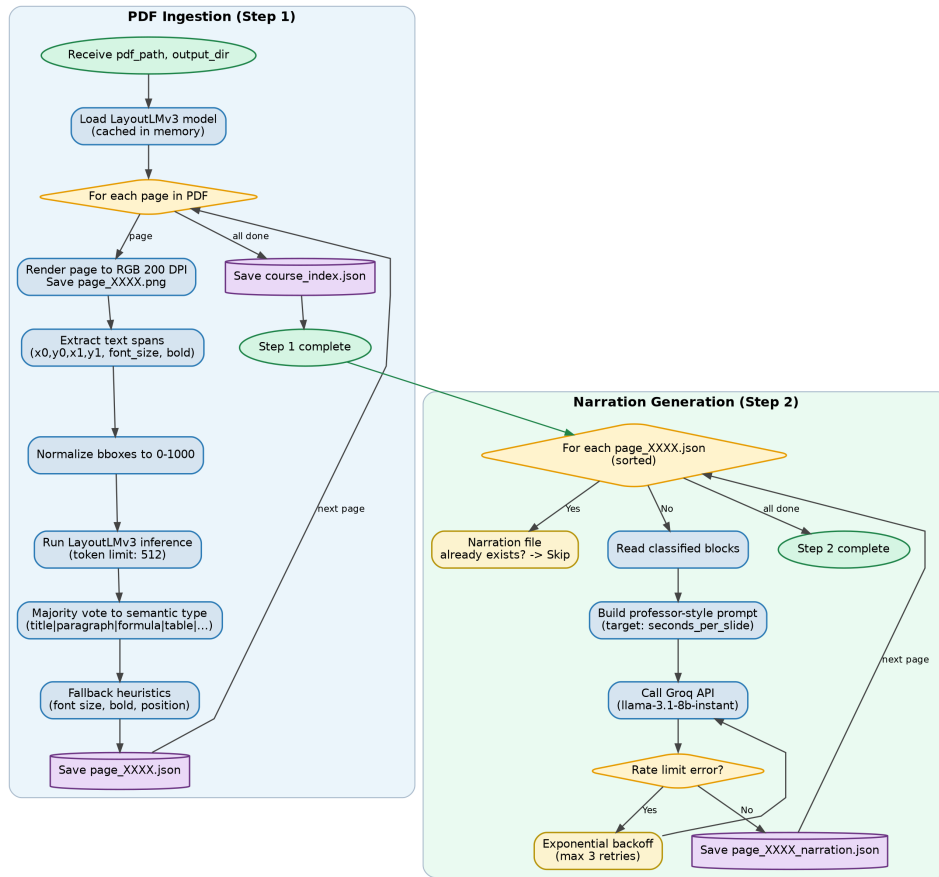


Figure 3.4: PDF Ingestion & Narration Generation Module

3.3.4 Module 4: RAG Indexing, Query & ChromaDB Seeding Module

- Purpose:** Provides semantic search and question-answering capabilities over course content, following the Retrieval-Augmented Generation (RAG) paradigm [32]. The seeding sub-module populates the shared ChromaDB [33] vector database with the OSSU Core Computer Science curriculum [34], enabling answers to general CS questions before any PDF is uploaded; seeding runs automatically on first use if the database does not yet exist. The indexing sub-module chunks and embeds narration text per course into the same database. The query sub-module retrieves relevant chunks at student question time and synthesises an answer via Gemini [30].
- Inputs:**
 - Seeding:* OSSU README.md [34] Core Computer Science section fetched at runtime from GitHub
 - Indexing:* output_dir with page_XXXX_narration.json files; course_i

d integer

- *Query*: POST/api/ai-courses/<id>/ask/ {question}; POST/api/ai-courses/<id>/reformulate/ {question,previous_answer}

- **Outputs:**

- `chroma_ossu_db/` — persistent ChromaDB directory with OSSU chunks
- ChromaDB populated with per-course narration chunks; `chroma_indexed.marker` written on completion
- `ask/`: {answer:"..."}
- `reformulate/`: {answer:"..."}

- **Dependencies:**

- *Internal*: `page_XXXX_narration.json` files (output of Module 3); `Functionalities/RAG/chroma_ossu_db/` shared ChromaDB instance
- *External*: `chromadb` [33]; `langchain-chroma`; `langchain-huggingface` with `all-MiniLM-L6-v2` [26] (80 MB local model); `langchain-google-genai` with `gemini-2.5-flash` [30]; `langchain-core` [35] (PromptTemplate, RunnablePassthrough, RunnableParallel, RunnableLambda, StrOutputParser); `langchain-text-splitters` (chunk size 700, overlap 100); `requests` (OSSU download)

- **Flow Description:** *Seeding*: Before every query or indexing operation, `_ensure_ossu_db_seeded()` checks for the presence of `chroma.sqlite3` in the database directory. If absent, the OSSU README.md is downloaded from GitHub, the *Core Computer Science* section is extracted via regex (falling back to the full document if the structure has changed), split into 700-character chunks with 100-character overlap, embedded with `all-MiniLM-L6-v2`, and persisted to `chroma_ossu_db/`. *Indexing (per course)*: If `chroma_indexed.marker` already exists the step is skipped. Otherwise all narration JSON files are loaded, split into chunks prefixed with `[Course{id}|Page{n}]`, embedded, and added to the shared ChromaDB collection; the marker file is then written. *Query (ask/)*: The student's question is embedded, a top- k ($k = 4$) similarity search retrieves the most relevant chunks, a prompt combining context and question is sent to `gemini-2.5-flash`, and the answer is returned as {answer}. *Query (reformulate/)*: Receives {question,previous_answer}. Uses a distinct prompt instructing the model to re-explain the same concept using a simple everyday analogy, shorter sentences, and an encouraging tone. The same top- k retrieval is performed on the question, and the simplified explanation is returned as {answer}. The flow is illustrated in Figure 3.5.

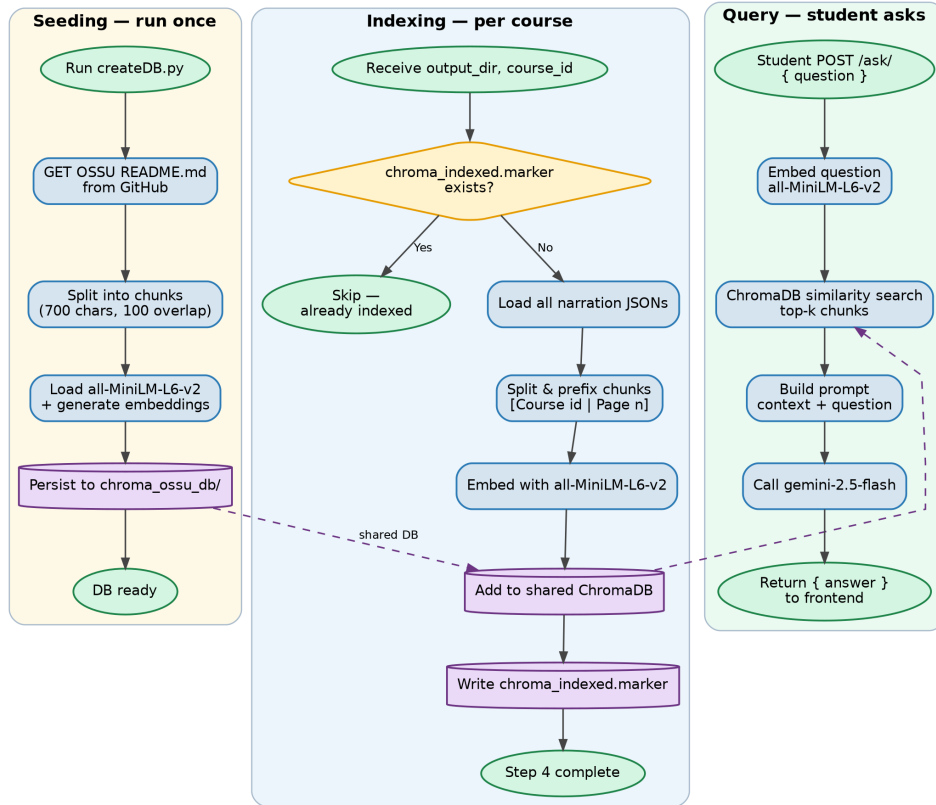


Figure 3.5: RAG Indexing, Query & ChromaDB Seeding Module

3.3.5 Module 5: Concept Animation Module

- **Purpose:** Generates short educational Manim animations on demand when a student requests a visual explanation of a concept. Gemini produces the Python Manim scene code; the Manim CLI renders it into an MP4 video. Results are cached by concept key so repeated requests for the same concept are served instantly without re-generation.
- **Inputs:**
 - `POST/api/ai-courses/<id>/animations/ {concept:"FourierTransform"}`
 - `ConceptAnimation` DB record (id, concept, concept_key, ai_course)
- **Outputs:**
 - `ConceptAnimation.status`: `PENDING` → `GENERATING` → `RENDERING` → `READY` | `ERROR`
 - `ConceptAnimation.video_file`: relative path to .mp4 in media/

- GET/animations/<id>/status/ returns {id,concept,status,video_url,error,retry_count,created_at}
- **Dependencies:**
 - *Internal:* api/models/ai.py (ConceptAnimation model); media/animations/anim_<id>/ working directory per animation
 - *External:* google-genai SDK with gemini-2.5-flash [30]; Manim Community CLI [21] (must be on PATH); Python subprocess for manim-ql--media_dir<anim_dir>scene.pyConceptScene; threading.Thread (daemon thread per animation)
- **Flow Description:** The student clicks *Animate* on a concept, triggering POST/api/ai-courses/<id>/animations/. The concept string is normalised to a concept_key (strip().lower()); if a READY animation for the same (ai_course, concept_key) already exists in the database, the endpoint returns {animation_id,status:"READY",cached:true} and the frontend immediately switches to the Animations tab without polling. Otherwise a ConceptAnimation record is created and a daemon thread is launched. *Step 1 — GENERATING:* a prompt is sent to gemini-2.5-flash requesting a ConceptScene Manim class; markdown fences are stripped and scene.py is written to anim_<id>/. *Step 2 — RENDERING:* manim-ql is invoked via subprocess with a 180-second timeout. On success the resulting .mp4 path is saved to the database and the status is set to READY. On error or timeout the retry counter is incremented; if fewer than 3 attempts have been made the process restarts from the GENERATING step, otherwise the status is set to ERROR. The frontend polls every 4 seconds and switches to the Animations tab when READY is detected. The flow is illustrated in Figure 3.6.

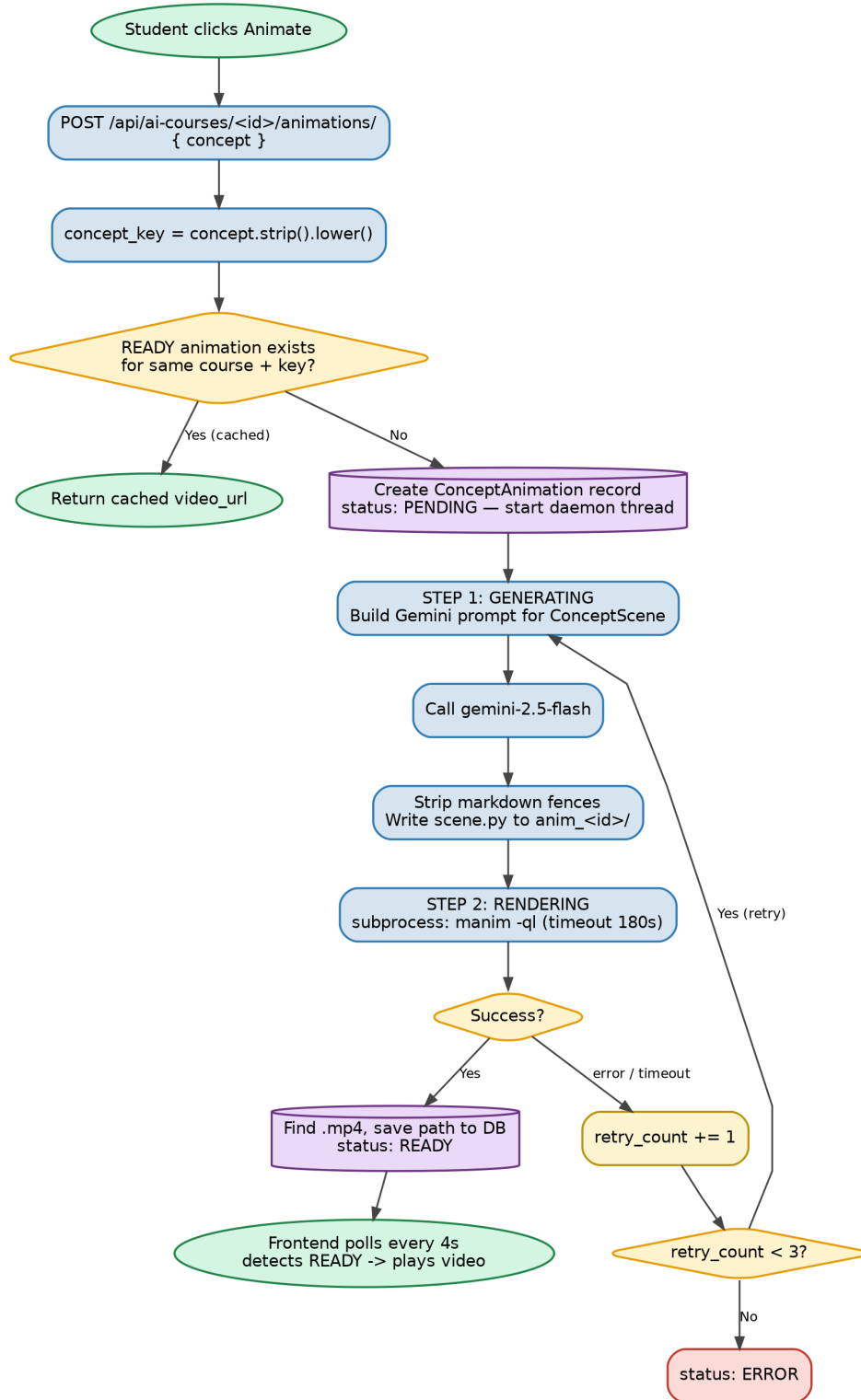


Figure 3.6: Concept Animation Module

3.3.6 Module 6: Quiz Module

- **Purpose:** Automatically generates a 20-question multiple-choice quiz from the narration content of an AI course. Teachers trigger generation via a dedicated endpoint; students submit answers and receive an instant score with a per-question breakdown; teachers can view per-student results.
- **Inputs:**
 - *Teacher:* POST/api/teacher/classes/<id>/ai-courses/<id>/generate-quiz/; GET/api/teacher/classes/<id>/ai-courses/<id>/quiz-results/
 - *Student:* GET/api/ai-courses/<id>/quiz/status/; GET/api/ai-courses/<id>/quiz/questions/; POST/api/ai-courses/<id>/quiz/attempt/ {answers: [0,2,1,3,...]}
- **Outputs:**
 - Quiz.questions — JSON array of 20 objects: {question,options[4],correct_index}
 - QuizAttempt — {attempt_id,score(%),correct_count,total,results:[{question,options,selected_index,correct_index,is_correct}]}
 - Teacher results — list of {attempt_id,student_name,student_email,score,correct_count,total,completed_at}
- **Dependencies:**
 - *Internal:* api/models/ai.py (Quiz OneToOneField to AICourse, QuizAttempt models); page_XXXX_narration.json files (max 40 000 chars total)
 - *External:* google-genai SDK with gemini-2.5-flash [30]; threading.Thread (daemon thread for generation); Python json
- **Flow Description:** *Generation:* The teacher triggers POST.../generate-quiz/; a Quiz record is created with status PENDING and a daemon thread is launched. All narration files are concatenated (truncated to 40 000 characters if needed) and sent to gemini-2.5-flash with a prompt requesting exactly 20 multiple-choice questions as a JSON array. The response is parsed and validated: at least 10 items must be present (fewer raises a ValueError and sets status to ERROR), the list is truncated to 20, and each item is checked for exactly 4 options and a correct_index in 0–3. On success the questions are stored and the status is set to READY; on validation failure the status is set to ERROR. *Student attempt:* The student submits answers via POST.../quiz/atte

mpt/. The answers list is validated (exactly 20 integers in 0–3); each answer is compared to the corresponding `correct_index`; `score = correct_count / 20 × 100` is computed, a `QuizAttempt` record is saved, and a result containing the score and a per-question breakdown is returned immediately. The flow is illustrated in Figure 3.7.

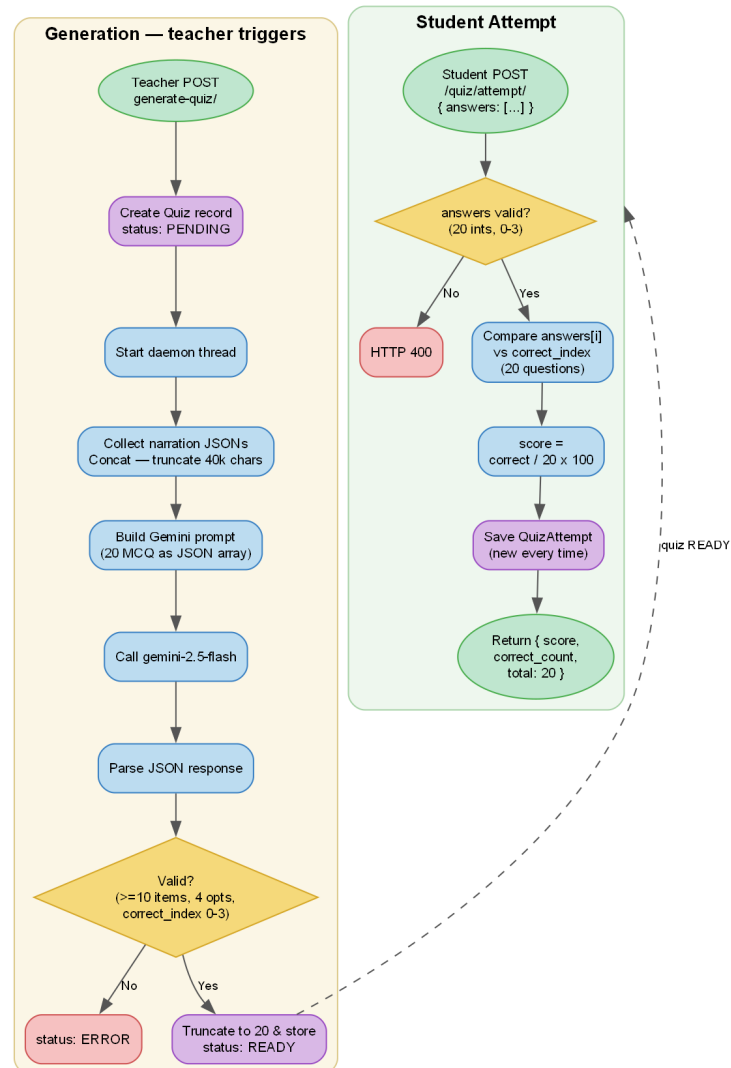


Figure 3.7: Quiz Module

3.4 Databases Design

PageSage AI uses two data stores that serve distinct but complementary roles. **PostgreSQL** handles all structured, transactional data — users, classes, enrollments,

assignments, submissions, AI courses, animations, and quizzes — providing the relational backbone of the platform. **ChromaDB** sits alongside it as a vector store, holding the embedded text chunks that power semantic search and the RAG question-answering pipeline. The two stores are loosely coupled by convention rather than by a foreign key: every chunk belonging to a course is prefixed with `[Course{id}|Page{n}]`, where the `id` matches the corresponding `AICourse` record in PostgreSQL, allowing retrieval to be scoped to a specific course without any cross-store referential constraint.

3.4.1 Schema Design

The relational schema is centred on the `users` table, which separates teachers from students through a `role` column (`student | teacher`) validated at the application layer. Every academic artifact — announcements, assignments, lectures, and AI courses — is scoped to a `Class` record, making the classroom the natural unit of organisation across the entire schema. Uploaded PDFs are represented by the `ai_courses` table (`AICourse` model), which tracks pipeline progress through `status` and `current_step` columns, stores the total page count after extraction, and keeps an `output_dir` path that ties the database record to both the generated files on disk and the corresponding ChromaDB chunks for that course.

3.4.2 Relational Model (PostgreSQL)

- **Identity:** all primary keys are auto-increment 64-bit integers, using Django's `BigAutoField` configured as the default in `core/settings/base.py`.
- **Integrity:** role and status enumerations are enforced at the application layer through Django `choices`; at the database level, `UNIQUE` constraints guard the three relationships where duplicates are semantically invalid — enrollments (`class_obj`, `student_email`), submissions (`assignment`, `student_email`), and concept animations (`ai_course`, `concept_key`).
- **Flexibility:** quiz content is stored as a `JSONB` column, allowing the question structure to evolve without requiring a schema migration.

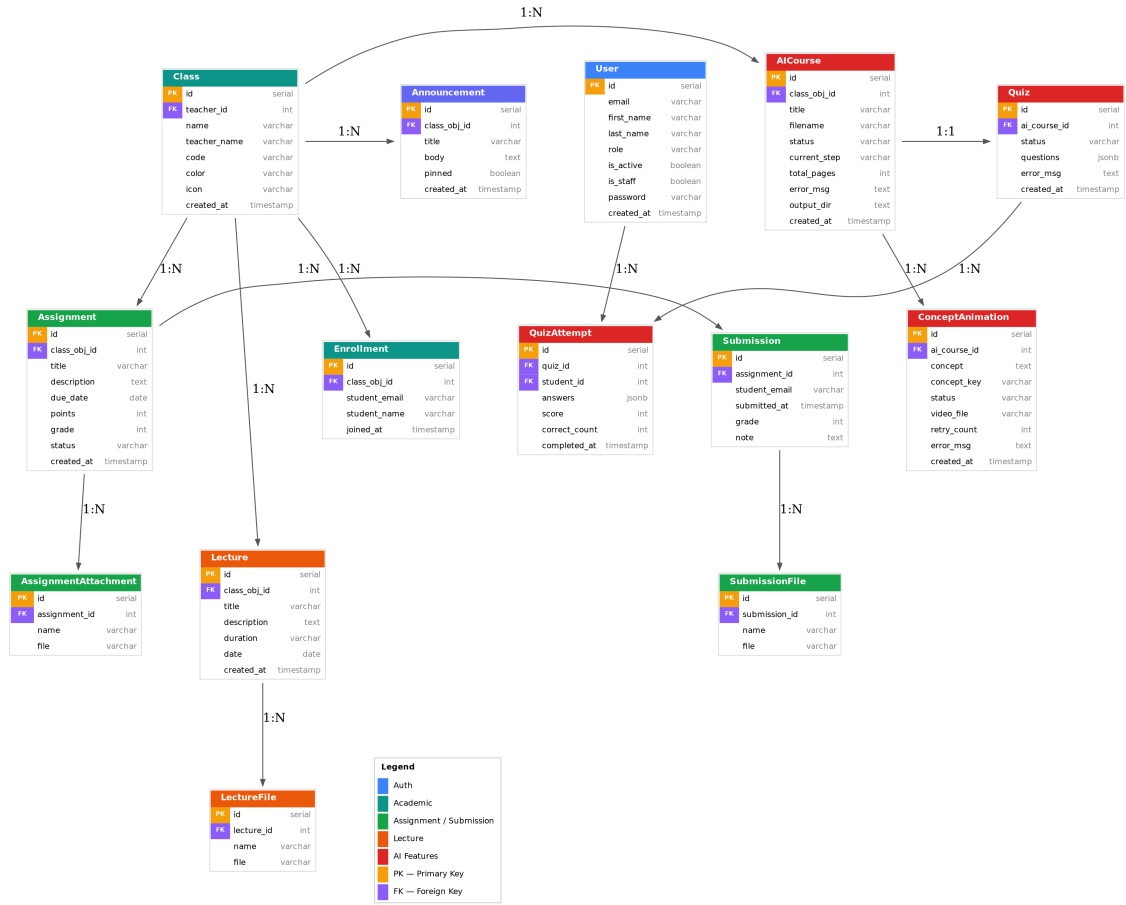


Figure 3.8: PostgreSQL entity-relationship diagram

3.4.3 Vector Store (ChromaDB)

ChromaDB [33] serves as the semantic memory of the platform, storing dense vector representations of text that the RAG pipeline retrieves at query time. The database is persisted to disk as a single SQLite file (`chroma.sqlite3`) located at `Functionalities/RAG/chroma_ossu_db/`, making it portable and independent of any external database server.

Collection design. Rather than maintaining separate collections per course or content type, the platform uses a single default collection in which two categories of content coexist: the Open Source Society University (OSSU) Core Computer Science curriculum [34], seeded once from GitHub at first use, and the per-page narration text generated by the AI pipeline for each uploaded course. The two are distinguished purely by a text prefix — OSSU chunks carry no prefix, while course chunks are

tagged `[Course{id}|Page{n}]` — which also allows the retriever to surface relevant material from both sources in a single similarity search, giving students access to general CS knowledge alongside their specific course content.

Embedding model. All text is embedded with `all-MiniLM-L6-v2` [26], a lightweight Sentence-BERT model producing 384-dimensional vectors. The model is loaded once per Django process and reused across all indexing and query operations to avoid repeated initialisation overhead.

Chunking. Text is split with LangChain’s `RecursiveCharacterTextSplitter` using a chunk size of 700 characters and an overlap of 100 characters, balancing retrieval granularity against context completeness.

Retrieval. At query time, the student’s question is embedded with the same model and the top $k = 4$ most similar chunks are retrieved by cosine similarity. These chunks are injected into the prompt sent to `gemini-2.5-flash` [30] for answer synthesis.

Auto-seeding. Before every query or indexing operation, the pipeline checks for the presence of `chroma.sqlite3`; if the file is absent, the OSSU curriculum is downloaded, the *Core Computer Science* section is extracted via regex, chunked, embedded, and persisted automatically — ensuring the database is always ready without manual setup.

4 System Evaluation

4.1 API Performance Testing

4.2 Load Testing

4.3 Pipeline Reliability

4.4 Security Testing

5 Empirical Evaluation

5.1 Overview

The transition from a functional software prototype to a validated educational tool requires more than technical correctness — it demands empirical evidence that the system produces meaningful improvements in the learning outcomes it targets. Wohlin et al. [36] argue that pilot studies occupy a critical role in software

evaluation, providing preliminary evidence of effect direction and magnitude before the investment required by large-scale controlled experiments is justified. In the context of educational software specifically, where the primary outcome of interest — genuine learning — is inherently subjective and confounded by individual differences in prior knowledge, motivation, and learning style, Leppink et al. [37] demonstrated that validated psychometric instruments offer a standardised method for quantifying otherwise unobservable constructs such as cognitive load and schema formation.

The particular importance of empirical evaluation in AI-powered educational systems appears from the gap between functional capability and pedagogical effectiveness. A system that correctly generates narration, animations, and quiz questions may still impose unnecessary cognitive burden on learners, fail to produce measurable learning gains, or simply not be adopted by the students it is designed to serve. Mustafa et al. [38], in a bibliometric analysis of 725 AI-in-education publications, identified student engagement, perceived usefulness, and cognitive accessibility as the dimensions most consistently linked to positive learning outcomes when AI tools are evaluated empirically. Frontiers in Education [39] further confirmed in a systematic review of AI in higher education that platforms which fail to achieve student adoption — regardless of technical quality — produce no measurable benefit to learning outcomes. Without empirical measurement of these dimensions, a system cannot be considered validated, regardless of the sophistication of its underlying architecture.

This chapter presents a between-subjects pilot study comparing PageSageAI against NotebookLM — identified in Chapter 2 (Table 2.1) as the most functionally comparable existing platform — across three validated psychometric dimensions whose selection is directly motivated by the architectural differences between the two systems.

Extraneous Cognitive Load (ECL), operationalised by Leppink et al. [37] as the processing burden imposed by the instructional design rather than by the content itself, targets the central architectural difference between the two platforms: NotebookLM delivers a fixed, non-interactive audio-visual summary, while PageSageAI delivers page-synchronised vocal narration with on-demand visual explanations — a difference that should manifest directly in the cognitive effort required to follow each presentation.

Germane Cognitive Load (GCL), from the same instrument, captures the productive cognitive work that directly predicts durable learning — schema construction and concept integration — making it the dimension most likely to reveal whether PageSageAI’s interactive and multimodal design translates into measurable learning gains beyond what a passive overview can achieve.

Technology Acceptance (TAM), grounded in Davis [40], addresses a fundamental constraint of any educational platform: one that students do not adopt produces no benefit regardless of its technical quality. Higher acceptance scores and stronger intention to reuse the platform would therefore provide direct evidence of PageSageAI’s viability as a production-ready educational tool, beyond what architectural analysis

alone can establish.

Together, ECL, GCL, and TAM form a coherent evaluation framework that covers the full instructional lifecycle: how the content is delivered (ECL), whether that delivery produces learning (GCL), and whether students would engage with the platform again (TAM)

5.2 Study Objectives

This pilot study pursues three specific empirical objectives, each corresponding directly to a validated measurement dimension and to a design decision described in Chapter 3.

The first objective is to determine whether PageSageAI’s page-synchronised vocal narration and on-demand animation pipeline reduce the extraneous cognitive load perceived by learners compared to NotebookLM’s static audio-visual overview, as measured by the ECL subscale of the Cognitive Load Scale [37].

The second objective is to determine whether the interactive and multimodal nature of PageSageAI’s teaching session — combining RAG-grounded question answering with Manim-generated concept visualisations — produces higher germane learning outcomes than a passive single-channel overview, as measured by the GCL subscale of the same instrument.

The third objective is to assess whether participants exposed to PageSageAI report higher technology acceptance and stronger intention to reuse a similar platform, as measured by items adapted from Davis’s Technology Acceptance Model [40], providing preliminary evidence for the system’s viability beyond a research prototype.

To isolate the effect of presentation format on these three outcomes, a between-subjects experimental design was adopted, in which participants were independently assigned to one of two conditions: exposure to a NotebookLM-generated video overview or exposure to a PageSageAI teaching session, both derived from the same ten-page source document on Gradient Descent. This design eliminates the carry-over and order effects that would confound a within-subjects comparison: had participants viewed both platforms sequentially, familiarity gained from the first exposure would have inflated performance scores on the second, making it impossible to attribute observed differences to the platform’s presentation quality alone.

The evaluation framework aligns with the usability and learnability quality characteristics defined in ISO/IEC 25010 [25]. Given the pilot nature of the study and the total sample size of $n = 20$, the primary goal is to assess the direction and plausibility of observed effects rather than to establish definitive statistical conclusions.

5.3 Participants

A total of twenty participants took part in the study, equally divided across the two conditions: ten assigned to the NotebookLM group and ten to the PageSageAI group. Participants were recruited through convenience sampling via digital distribution channels, with almost no prior knowledge of the research study disclosed to them. All participants confirmed the truthfulness and accuracy of their responses via an explicit consent statement at the start of the questionnaire; one participant in the PageSageAI group omitted this confirmation but provided complete responses to all substantive items and was retained in the analysis.

Table 5.1 summarises the demographic profile of the two groups. The sample was predominantly composed of university students and high school students, with a majority (eighteen out of twenty participants) falling into the 18–25 age bracket. Education-level distributions were comparable across conditions: the NotebookLM group comprised eight undergraduate (BSc) students and two high school students, while the PageSageAI group comprised seven undergraduate (BSc) students, two high school students, and one graduate (MSc) student.

Prior familiarity with the study topic (Gradient Descent, self-rated on a scale from 1 to 10) was slightly higher in the PageSageAI group ($M = 3.30$, $SD = 1.06$) compared to the NotebookLM group ($M = 2.80$, $SD = 0.79$). This difference is modest and, if anything, creates a slight disadvantage for the PageSageAI group on learning measures, since participants with greater prior knowledge tend to report lower perceived learning gains. The familiarity distributions are therefore not expected to confound the primary comparisons in favour of PageSageAI.

Table 5.1: Demographic summary by condition

Characteristic	NotebookLM ($n = 10$)	PageSageAI ($n = 10$)
Age: 18–25	10	9
Age: > 25	0	1
Education: BSc programme	8	7
Education: Highschool	2	2
Education: MSc programme	0	1
Prior familiarity (1–10), $M(SD)$	2.80 (0.79)	3.30 (1.06)

5.4 Materials and Instruments

5.4.1 Source Material

Both conditions were derived from a common ten-page English-language course document on Gradient Descent, covering the cost function, the gradient, the learning rate, the mountain-descent analogy, batch versus stochastic variants, convergence

criteria, and a Python implementation example. The use of an identical source document across both conditions ensures that any differences in participant outcomes can be attributed to the presentation format rather than to differences in content.

5.4.2 Instructional conditions

NotebookLM condition. The source PDF was uploaded to NotebookLM, powered by Gemini 1.5 Pro [30], which generated a static audio-visual overview of approximately 8 minutes. The video is a non-interactive summary presentation: participants could view it sequentially but could not pause to ask questions, request concept clarification, or receive on-demand visual explanations of abstract content. The generation process required approximately 17:33 minutes, measured from the moment generation was triggered to the moment the video became available for viewing, recorded via screen-capture with timestamp.

PageSageAI condition. The same source PDF was processed through the full PageSageAI pipeline described in Chapter 3, producing a structured vocal teaching session of approximately 16:08 minutes. The recorded session demonstrated the platform’s complete feature set: page-synchronised vocal narration generated from LayoutLMv3-classified content blocks and an on-demand Manim animation of the gradient descent optimisation trajectory. The session was recorded and delivered to participants as a video. Total pipeline processing time—PDF ingestion, narration generation, TTS synthesis via Deepgram Aura-2, and ChromaDB indexing—was approximately 15:20.

5.4.3 Questionnaire and Procedure

The questionnaire was administered digitally via Google Forms [41] immediately following each participant’s viewing session. It comprised three sections: (1) demographic information, (2) an Intrinsic Cognitive Load (ICL) [37] baseline item assessing perceived content complexity, (3) and the three primary dependent-variable instruments (ECL, GCL, TAM).

Each of the three primary instruments was presented as a forced-choice item in which participants selected the single statement from a set of five that best described their experience of the course material. The score for each instrument was the ordinal index (1–5) of the selected statement, yielding a scale consistent with the five-item structure of the validated source instruments.

Extraneous Cognitive Load (ECL) was measured using five items adapted from the Cognitive Load Scale developed and validated by Leppink et al. [37]. ECL captures the cognitive burden imposed by the instructional presentation rather than by the inherent difficulty of the material; higher scores indicate worse presentation quality.

Germane Cognitive Load (GCL) was measured using five further items from the same validated instrument [37]. GCL reflects the mental effort effectively

invested in understanding and schema formation—the productive dimension of cognitive processing that results in durable learning; higher scores indicate greater perceived learning.

Technology Acceptance (TAM) was measured using five items adapted from Davis’s Technology Acceptance Model [40], capturing perceived usefulness, ease of use, and behavioural intention to reuse the platform; higher scores indicate more positive acceptance.

Participants were recruited and assigned to conditions via independent distribution of two separate questionnaire links, each pre-labelled with the corresponding platform. Participants in the NotebookLM group were directed to watch the NotebookLM-generated video overview of Gradient Descent before completing the questionnaire; participants in the PageSageAI group were directed to watch the PageSageAI teaching-session recording. Neither group had prior access to the other group’s stimulus material, and no communication between participants from different conditions was permitted during the data-collection period.

Upon completing the video, participants accessed the Google Forms [41] questionnaire and confirmed via an explicit consent statement that their answers truthfully represented their own perceptions and experience of the course material. They then selected the ICL item that best described the perceived complexity of the course content, followed by the ECL, GCL, and TAM items. The estimated total time per participant, including video viewing and questionnaire completion, was 25–35 minutes.

5.5 Results

5.5.1 Descriptive Statistics

Table 5.2 presents the mean, standard deviation, and median scores for each dependent variable by condition. Across all three variables, the observed means were in the direction predicted by the study hypothesis: the PageSageAI group reported lower ECL ($M = 2.60$ vs. $M = 3.90$, i.e. better presentation quality), higher GCL ($M = 3.70$ vs. $M = 1.40$), and higher TAM ($M = 4.00$ vs. $M = 2.10$) relative to the NotebookLM group.

Table 5.2: Descriptive statistics by condition

Variable	NotebookLM ($n = 10$)			PageSageAI ($n = 10$)		
	M	SD	Mdn	M	SD	Mdn
ECL [1–5, lower=better]	3.90	1.29	4.0	2.60	1.71	2.0
GCL [1–5, higher=better]	1.40	0.52	1.0	3.70	1.57	4.0
TAM [1–5, higher=better]	2.10	1.29	2.0	4.00	1.41	4.5

Note: ECL/GCL adapted from Leppink et al. (2013); TAM adapted from Davis (1989).

The ICL item, included as a baseline check on perceived content difficulty [37], showed near-identical response distributions across the two groups: both groups described the material as inherently complex at comparable rates, confirming that the two groups perceived the intrinsic difficulty of the source material similarly and that the observed ECL/GCL differences are not attributable to differences in perceived content complexity.

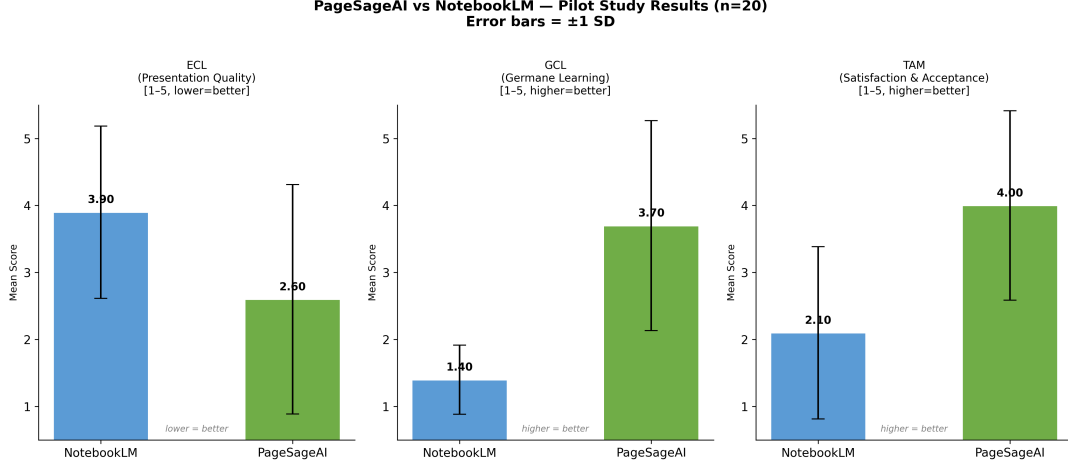


Figure 5.1: Mean scores and standard deviations across all three dependent variables (ECL, GCL, TAM), showing the direction of observed differences favoring PageSageAI on all measures.

5.6 Discussion

5.6.1 Interpretation of Results

The pilot study results provide preliminary support for all three directional hypotheses: PageSageAI produced lower extraneous cognitive load, higher germane learning, and higher technology acceptance compared to NotebookLM on the same source material. Two of the three differences reached statistical significance despite a total sample of only twenty participants showing also a meaningful outcome given the power constraints inherent to pilot studies at this scale [36] — and all three means were in the predicted direction, yielding a consistent picture across the full evaluation framework.

The GCL difference is the most significant finding. A mean of 1.40 in the NotebookLM group indicates that participants predominantly selected the lowest-order learning statement, reflecting only minimal improvement in understanding. The PageSageAI group’s mean of 3.70 suggests that participants experienced a qualitatively different learning process, with the majority reporting that concepts were clarified, that they could explain Gradient Descent in their own words, or

that the material helped them build a coherent conceptual structure. This outcome does not predict only the differences between NotebookLM and PageSageAI but it aligns with Mayer’s Multimedia Learning Theory [7], which predicts superior learning from combinations of narration and dynamic animation over audio-only or text-only presentation.

The TAM result reflects a substantial gap in perceived usefulness and behavioural intention. The NotebookLM group’s mean of 2.10—below the scale midpoint—indicates that many participants did not find the static format useful or would not seek to reuse a similar tool, whereas the PageSageAI group’s mean of 4.00 indicates broadly positive acceptance. This is consistent with the qualitative characterisation in Section 2.1: NotebookLM’s audio-visual overviews are non-interactive by design, which limits their usefulness for learners who need to ask questions mid-lecture, revisit specific concepts, or see abstract processes demonstrated dynamically.

The ECL result was directionally consistent with the hypothesis ($M_{NB} = 3.90$ vs. $M_{PSA} = 2.60$) but did not reach statistical significance ($p = .110$). The higher variance in the PageSageAI group ($SD = 1.71$ vs. $SD = 1.29$) suggests that participants responded differently to the multimodal format: while most found it clearer than the static overview, a minority appeared to find the combination of vocal narration and animation more demanding rather than less. Mayer [42] documents this pattern — learners with lower working memory capacity sometimes experience higher rather than lower cognitive load when processing coordinated verbal and visual streams simultaneously — which offers a plausible explanation for the increased spread of ECL scores in the PageSageAI group and the resulting non-significant between-group contrast.

5.6.2 Limitations

Several methodological constraints limit the strength of conclusions that can be drawn from this study and must be acknowledged before interpreting its findings.

Sample size and statistical power. The most significant constraint is the sample size of $n = 20$. A pilot study at this scale is designed to assess the plausibility and direction of effects, not to serve as a definitive basis for generalisation [36]. Statistical power at $n = 10$ per group is insufficient to reliably detect small-to-medium effect sizes, and the non-significant ECL result ($p = .110$) cannot be interpreted as evidence of no effect — it may simply reflect an underpowered test on a measure with known reliability variability in multimodal contexts [43]. The results presented here should therefore be read as a signal, not a verdict.

Single concept and limited generalisability. The study evaluated a single STEM concept — Gradient Descent — which is inherently visual and procedural, making it particularly well-suited to PageSageAI’s animation pipeline and well-matched to the meta-analytic evidence on dynamic visualisations for algorithmic content [8]. It is not known whether the observed advantages generalise to less visual or more linguistically complex STEM domains — such as formal logic, probability

theory, or systems design — where the Manim module would contribute less directly to comprehension. A single-concept evaluation cannot therefore support broad claims about platform effectiveness across the full breadth of university STEM curricula.

5.6.3 Conclusions

With these constraints in mind, the results of this pilot study offer preliminary but consistent empirical evidence that PageSageAI produces a measurably superior instructional experience compared to NotebookLM when delivering the same STEM content. Two of the three differences reached statistical significance — GCL ($p = .005$) and TAM ($p = .014$) — and all three means were in the predicted direction, forming a coherent pattern rather than an isolated finding. Given the limitations discussed above, these results constitute a directional signal rather than a conclusive proof: they establish that the hypothesised effects are real and detectable, and that replication with a larger and more diverse sample would be needed.

What the data collectively suggest is that the architectural decisions described in Chapter 3 — combining RAG-grounded interaction, page-synchronised vocal narration, and on-demand Manim animation into a single coherent platform — translate into a learning experience that participants found both more effective and more worth returning to. This is precisely the gap that Chapter 2 identified in the current landscape: not a lack of AI educational tools, but a lack of tools that integrate all dimensions of the teaching lifecycle into one environment. The results of this study, modest in scale but consistent in direction, suggest that PageSageAI represents a meaningful step towards closing that gap.

6 Conclusion and Future Work

6.1 Conclusion

TO DO

6.2 Future Work

TO DO

References

- [1] A. Harry and Sayudin, “Role of AI in education,” *INJURY: Interdisciplinary Journal and Humanity*, vol. 2, no. 3, pp. 260–268, 2023.
- [2] S. Nature, “From algorithms to tutors: tracing the evolution of generative AI in education,” *SN Social Sciences*, 2026.
- [3] F. Mustafa, H. T. M. Nguyen, and X. Gao, “Artificial intelligence in education (AIEd): Publication patterns, keywords, and research focuses,” *Information*, vol. 16, no. 9, p. 725, 2025.
- [4] O. Zawacki-Richter *et al.*, “Personalised learning through artificial intelligence: A systematic review,” *Computers & Education*, vol. 210, 2025.
- [5] D. J. Nicol and D. Macfarlane-Dick, “Formative assessment and self-regulated learning: a model and seven principles of good feedback practice,” *Studies in Higher Education*, vol. 31, no. 2, pp. 199–218, 2006.
- [6] F. in Education, “Artificial intelligence in higher education: a systematic review of its impact on student engagement,” *Frontiers in Education*, 2025.
- [7] R. E. Mayer, *Multimedia Learning*, 3rd ed. Cambridge University Press, 2020.
- [8] R. K. Lowe, “Dynamic visualizations and learning: getting to the difficult questions,” *Learning and Instruction*, vol. 14, no. 3, pp. 257–262, 2004.
- [9] P. Janků and C. Brom, “Influence of 3D models and animations on students in natural subjects,” *International Journal of STEM Education*, vol. 9, no. 1, p. 56, 2022.
- [10] OECD, “How do student-teacher ratios and class sizes vary across education levels up to upper secondary education?” OECD Publishing, Tech. Rep., 2025. [Online]. Available: <https://www.oecd.org/en/publications/education-at-a-glance-2025>
- [11] L. Zhang *et al.*, “The impact of self-directed learning experience and course experience on learning satisfaction of university students in blended learning environments,” *Frontiers in Psychology*, vol. 14, 2023.
- [12] S. Elnaffar, F. Rashidi, and A. Z. Abualkishik, “Teaching with AI: A systematic review of chatbots, generative tools, and tutoring systems in programming education,” *International Journal of Learning, Teaching and Educational Research*, vol. 25, no. 1, pp. 1–28, 2026.
- [13] F. in Education, “Artificial intelligence and learner autonomy: a meta-analysis of self-regulated and self-directed learning,” *Frontiers in Education*, 2025.

- [14] Education Technology Insights, “AI as architect of integrated learning,” 2025. [Online]. Available: <https://artificial-intelligence.educationtechnologyinsights.com/news/ai-as-architect-of-integrated-learning-nid-3492.html>
- [15] F. in Education, “Artificial intelligence in higher education, opportunities, and challenges: a review,” *Frontiers in Education*, 2025.
- [16] Khan Academy, “Meet Khanmigo: Khan Academy’s AI-powered teaching assistant & tutor,” 2024, accessed: 2026. [Online]. Available: <https://www.khanmigo.ai/>
- [17] W. A. Sahlman, A. M. Ciechanover, and E. Grandjean, “Khanmigo: Revolutionizing learning with GenAI,” Harvard Business School, Tech. Rep. Case 824-059, 2023.
- [18] Google, “NotebookLM: AI research & thinking partner,” 2024, accessed: 2026. [Online]. Available: <https://notebooklm.google/>
- [19] M. Figueiredo *et al.*, “The potential of Google NotebookLM for teaching and learning,” *ResearchGate Preprint*, 2025. [Online]. Available: <https://www.researchgate.net/publication/393965130>
- [20] SchoolAI, “SchoolAI: AI teaching assistant for K-12 classrooms,” 2024, accessed: 2026. [Online]. Available: <https://schoolai.com/>
- [21] The Manim Community Developers, “Manim – mathematical animation framework,” 2024. [Online]. Available: <https://www.manim.community/>
- [22] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.
- [23] L. Chung, B. A. Nixon, E. Yu, and J. Mylopoulos, *Non-Functional Requirements in Software Engineering*. Kluwer Academic Publishers, 2000.
- [24] J. Eckhardt, A. Vogelsang, and D. M. Fernández, “Are “non-functional” requirements really non-functional? an investigation of non-functional requirements in practice,” in *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. IEEE, 2016, pp. 832–842.
- [25] ISO/IEC, “Systems and software quality requirements and evaluation (SQuaRE) — system and software quality models,” International Organization for Standardization, Tech. Rep. ISO/IEC 25010:2011, 2011.
- [26] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, 2019, pp. 3982–3992.

- [27] Artifex Software, Inc., “PyMuPDF: High-performance PDF and document manipulation library,” 2024, accessed: 2026. [Online]. Available: <https://github.com/pymupdf/PyMuPDF>
- [28] T. Wolf, L. Debut, V. Sanh *et al.*, “HuggingFace’s Transformers: State-of-the-art natural language processing,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 2020, pp. 38–45.
- [29] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei, “LayoutLMv3: Pre-training for document AI with unified text and image masking,” in *Proceedings of the 30th ACM International Conference on Multimedia*. ACM, 2022, pp. 4083–4091.
- [30] Gemini Team, Google, “Gemini: A family of highly capable multimodal models,” *arXiv preprint arXiv:2312.11805*, 2023.
- [31] A. Grattafiori *et al.*, “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [32] P. Lewis, E. Perez, A. Piktus *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [33] Chroma, “Chroma: the AI-native open-source embedding database,” 2022, accessed: 2026. [Online]. Available: <https://github.com/chroma-core/chroma>
- [34] Open Source Society University, “OSSU computer science curriculum,” 2024, accessed: 2026. [Online]. Available: <https://github.com/ossu/computer-science>
- [35] H. Chase, “LangChain,” 2022, accessed: 2026. [Online]. Available: <https://github.com/langchain-ai/langchain>
- [36] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.
- [37] J. Leppink, F. Paas, C. P. M. Van der Vleuten, T. Van Gog, and J. J. G. Van Merriënboer, “Development of an instrument for measuring different types of cognitive load,” *Behavior Research Methods*, vol. 45, no. 4, pp. 1058–1072, 2013.
- [38] F. Mustafa, H. T. M. Nguyen, and X. Gao, “Artificial intelligence in education (AIED): Publication patterns, keywords, and research focuses,” *Information*, vol. 16, no. 9, p. 725, 2025.
- [39] Frontiers in Education, “Artificial intelligence in higher education: a systematic review of its impact on student engagement,” *Frontiers in Education*, 2025.

- [40] F. D. Davis, “Perceived usefulness, perceived ease of use, and user acceptance of information technology,” *MIS Quarterly*, vol. 13, no. 3, pp. 319–340, 1989.
- [41] Google LLC, “Google forms: Free online surveys for collecting feedback,” 2024, accessed: 2026. [Online]. Available: <https://www.google.com/forms/about/>
- [42] R. E. Mayer, *Multimedia Learning*, 3rd ed. Cambridge University Press, 2020.
- [43] M. S. Andersen and G. Makransky, “The validation and further development of the multidimensional cognitive load scale for physical and online lectures (MCLS-POL),” *Frontiers in Psychology*, vol. 12, 2021.